

Comprendre script.js

Guide detaille pour relire le JavaScript du portfolio, partie par partie, avec une logique de niveau junior.

Objectif du document : comprendre ce que fait chaque bloc, dans quel ordre JavaScript le lit, et pourquoi certaines lignes sont placees a cet endroit.

1. La grande idee du script

Ton fichier ne recharge pas toute la page quand on clique sur la navigation. Il garde la meme page HTML, puis il change seulement trois choses : le titre, le paragraphe et l'image. Pour la partie Projets, il affiche aussi les deux blocs de liens.

Le script est range dans un grand evenement **DOMContentLoaded**. Cela veut dire : attends que le HTML soit charge avant de chercher les elements.

```
document.addEventListener('DOMContentLoaded', function () {  
    // Tout le script principal est ici.  
});
```

A retenir : le navigateur lit d'abord le HTML. Ensuite seulement, ce code peut recuperer les elements comme le titre, le paragraphe, l'image et la navigation.

2. Le prechargement des images

Cette partie sert a eviter le petit saut visuel au premier clic. On donne au navigateur la liste des images importantes pour qu'il les charge en avance.

```
const imagesAPrecharger = [  
    'images/villedenuit1.jpg',  
    'images/competences-v2.png',  
    'images/liberte.jpg'  
];
```

Le tableau contient simplement des chemins d'images. Chaque ligne est une adresse vers un fichier du dossier images.

```
imagesAPrecharger.forEach(function (sourceImage) {  
    const imagePrechargee = new Image();  
    imagePrechargee.src = sourceImage;  
});
```

Comment le lire

forEach veut dire : pour chaque element du tableau, execute la fonction. Le parametre **sourceImage** recoit une valeur differente a chaque tour.

- Premier tour : sourceImage vaut 'images/villedenuit1.jpg'.
- Deuxieme tour : sourceImage vaut 'images/competences-v2.png'.
- Troisieme tour : sourceImage vaut 'images/liberte.jpg'.

new Image() cree un nouvel objet image. Il ressemble a une balise img creee en JavaScript, mais il n'est pas ajoute dans le HTML.

imagePrechargee.src = sourceImage donne une source a cette image. Des qu'une image recoit un src, le navigateur commence a charger le fichier.

Elle est chargee, mais pas affichee, car on ne fait jamais `document.body.appendChild(imagePrechargee)`. Elle reste donc en memoire.

3. Recuperer les elements HTML

Cette partie fait le lien entre ton fichier JavaScript et ton HTML. Avant de modifier quelque chose dans la page, JavaScript doit d'abord trouver les elements concernes.

```
const liensNavigation = document.querySelectorAll('nav ul li a');
const titre = document.getElementById('titreh2');
const paragraphe = document.getElementById('textep');
const image = document.getElementById('monImage');
const liensProjets = document.getElementById('projetsLiens');
```

querySelectorAll

document.querySelectorAll('nav ul li a') cherche tous les liens a l'interieur de la navigation. Comme il y a plusieurs liens, JavaScript renvoie une liste.

getElementById

document.getElementById cherche un seul element precis grace a son id. Par exemple, **titreh2** correspond au h2 de ton HTML.

Pourquoi les recuperer une seule fois ? Parce que ces elements existent deja dans la page. On les garde dans des variables, puis on les reutilise au clic.

4. Preparer les textes et les images

Ici, on prepare les contenus qui pourront etre affiches quand on clique sur Accueil, Competences ou Projets.

```
const texteAccueil = paragraphe.innerHTML;
const imageAccueil = 'images/villedenuit1.jpg';
const descriptionAccueil = image.alt;
```

texteAccueil = paragraphe.innerHTML recupere le texte d'accueil directement depuis le HTML. C'est pratique, car ce texte n'est pas duplique dans le JavaScript.

On utilise **innerHTML** au lieu de **textContent** parce que ton paragraphe contient une balise **
. **innerHTML garde cette balise, donc le saut de ligne reste present.

```
const texteCompetences = `Mes compétences représentent...`;
const texteProjets = `Mes projets sont la partie concrète...`;
```

Les textes longs sont ranges dans des variables. Comme ca, le bloc du clic reste plus facile a lire. Au moment du clic, on choisit juste la bonne variable.

Les backticks ` permettent d'ecrire un texte sur plusieurs lignes. C'est ce qu'on appelle une template literal.

5. Donner un comportement aux liens de navigation

La navigation contient plusieurs liens. Il faut donc parcourir cette liste et ajouter un événement de clic sur chaque lien.

```
liensNavigation.forEach(function (lien) {  
    lien.addEventListener('click', function (event) {  
        event.preventDefault();  
        // Le reste du code du clic est ici.  
    });  
});
```

Le premier `forEach`

`liensNavigation.forEach` passe sur chaque lien de la nav. Le paramètre **lien** représente le lien en cours.

`addEventListener('click')`

`addEventListener` veut dire : écoute un événement. Ici, on écoute l'événement **click**. La fonction à l'intérieur sera lancée seulement quand l'utilisateur clique sur ce lien.

`event.preventDefault()`

Dans le HTML, les liens ont `href="#"`. Sans `preventDefault`, le navigateur essaierait quand même de suivre ce lien. Ici, on dit : ne fais pas le comportement normal du lien, c'est JavaScript qui gère le clic.

6. Savoir quelle section a été cliquée

```
const section = lien.textContent.trim();
```

`lien.textContent` récupère le texte visible du lien cliqué. Si tu cliques sur Projets, `section` vaudra 'Projets'.

`trim()` retire les espaces inutiles au début et à la fin. C'est une petite sécurité pour comparer un texte proprement.

7. Choisir le titre à afficher

```
let texteTitre = section;  
  
if (section === 'Accueil') {  
    texteTitre = 'Bienvenue';  
}
```

Au départ, le titre prend la même valeur que la section cliquée. Si on clique sur Compétences, le titre devient Compétences. Si on clique sur Projets, le titre devient Projets.

Le cas spécial, c'est Accueil. Dans la nav, le mot Accueil est naturel. Mais dans la page, tu veux afficher Bienvenue. C'est pour ça qu'il y a ce `if`.

8. Preparer le contenu par default

```
let texteParagraphe = texteAccueil;  
let sourceImage = imageAccueil;  
let descriptionImage = descriptionAccueil;
```

Ici, on part du principe que le contenu a afficher est celui de l'accueil. C'est le contenu par default.

Ensuite, si la section cliquee est Competences ou Projets, les if / else vont remplacer ces valeurs.

Cette technique est simple a lire : on prepare une valeur de base, puis on la modifie seulement si on est dans un autre cas.

9. Remplacer le contenu selon la section

```
if (section === 'Compétences') {  
  texteParagraphe = texteCompetences;  
  sourceImage = 'images/competences-v2.png';  
  descriptionImage = 'Logos des compétences web, full stack et IA';  
} else if (section === 'Projets') {  
  texteParagraphe = texteProjets;  
  sourceImage = 'images/liberté.jpg';  
  descriptionImage = 'Image de présentation des projets';  
}
```

Cette partie choisit les bonnes valeurs selon le lien clique. Elle ne change pas encore l'ecran. Elle prepare seulement les valeurs qui seront utilisees plus bas.

- Si section vaut Competences, on prend le texte et l'image des competences.
- Sinon, si section vaut Projets, on prend le texte et l'image des projets.
- Sinon, on garde les valeurs de l'accueil preparees juste avant.

10. Afficher ou cacher les liens projets

```
if (section === 'Projets') {  
  liensProjets.classList.add('visible');  
} else {  
  liensProjets.classList.remove('visible');  
}
```

Le bloc des projets n'a du sens que dans la section Projets. Donc si on est dans Projets, on ajoute la classe visible. Sinon, on l'enleve.

classList.add ajoute une classe CSS. **classList.remove** retire une classe CSS. Le JavaScript decide l'etat, et le CSS decide l'apparence.

11. L'animation du titre

```
titre.classList.add('fade-out');
```

Cette ligne ajoute une classe CSS au titre. Si la classe fade-out contient une animation CSS, le titre commence à disparaître.

```
titre.addEventListener('animationend', function handleTitleAnimationEnd() {  
    titre.textContent = texteTitre;  
    paragraphe.innerHTML = texteParagraphe;  
  
    titre.classList.remove('fade-out');  
    titre.classList.add('fade-in');  
  
    titre.removeEventListener('animationend', handleTitleAnimationEnd);  
});
```

animationend est un événement envoyé par le navigateur quand l'animation CSS est terminée. Ici, on attend que le titre ait fini de disparaître avant de changer son texte et le paragraphe.

La fonction s'appelle **handleTitleAnimationEnd** pour pouvoir la retirer avec `removeEventListener`. Si la fonction était anonyme, ce serait plus dur de la retirer proprement.

Ordre important : on lance fade-out, on attend la fin, on change le contenu, puis on lance fade-in. C'est ce qui rend la transition plus douce.

12. Retirer fade-in après l'animation

```
titre.addEventListener('animationend', function removeTitleFadeIn() {  
    titre.classList.remove('fade-in');  
    titre.removeEventListener('animationend', removeTitleFadeIn);  
});
```

Une fois que le titre est revenu avec fade-in, on retire la classe fade-in. Ça permet de repartir proprement au prochain clic.

C'est aussi pour ça qu'on retire l'écouteur avec `removeEventListener` : cette fonction ne doit servir qu'une seule fois pour cette animation.

13. L'animation de l'image

```
image.classList.remove('fade-in', 'fade-out');  
image.classList.add('fade-out');
```

Avant de lancer une nouvelle animation, on retire les anciennes classes. C'est une remise à zéro simple. Ensuite, on ajoute `fade-out` pour faire disparaître l'image actuelle.

```
setTimeout(function () {  
    // changement de l'image après 300 ms  
}, 300);
```

setTimeout demande à JavaScript d'attendre un peu avant d'exécuter une fonction. Ici, on attend 300 millisecondes, le temps que l'image commence ou finisse son `fade-out`.

300 ms = 0,3 seconde. Ce nombre doit rester cohérent avec la durée de l'animation CSS.

14. Charger la nouvelle image avant de l'afficher

```
const nouvelleImage = new Image();  
  
nouvelleImage.addEventListener('load', function () {  
    image.src = sourceImage;  
    image.alt = descriptionImage;  
});  
  
nouvelleImage.src = sourceImage;
```

Cette partie crée une image temporaire. Son rôle est de vérifier que le nouveau fichier est bien chargé avant de changer la vraie image visible dans la page.

load est un événement qui se déclenche quand le fichier image est chargé. Quand cet événement arrive, on peut remplacer l'image visible.

La ligne **`nouvelleImage.src = sourceImage`** est placée après `addEventListener`. Comme ça, l'écouteur `load` est déjà prêt avant que le chargement commence.

15. Faire reapparaître l'image

```
image.classList.remove('fade-out');
image.classList.add('fade-in');

setTimeout(function () {
    image.classList.remove('fade-in');
}, 300);
```

Une fois la nouvelle image chargée, on retire fade-out puis on ajoute fade-in. L'image réapparaît donc avec une animation.

Le deuxième setTimeout sert à retirer fade-in après l'animation. La classe ne reste pas bloquée sur l'image, ce qui aide à relancer correctement l'animation au clic suivant.

16. Exemple complet : clic sur Projets

Voici ce que tu peux imaginer quand tu cliques sur Projets :

- Le navigateur lance la fonction du clic.
- `event.preventDefault` empêche le lien # de bouger la page.
- section devient 'Projets'.
- `texteTitre` devient 'Projets'.
- Les valeurs par défaut sont d'abord celles de l'accueil.
- Le `else if Projets` remplace le texte, l'image et le alt.
- La classe visible est ajoutée au bloc des liens projets.
- Le titre disparaît avec fade-out.
- Quand l'animation du titre finit, le titre et le paragraphe changent.
- L'image actuelle disparaît.
- Après 300 ms, JavaScript prépare la nouvelle image.
- Quand cette nouvelle image est chargée, la vraie balise `img` est mise à jour.
- L'image réapparaît avec fade-in.

17. Les notions à revoir tranquillement

- Les tableaux et `forEach` : parcourir une liste élément par élément.
- Les fonctions callback : une fonction donnée à une autre fonction.
- Les objets et propriétés : `image.src`, `image.alt`, `classList`.
- Le DOM : récupérer et modifier des éléments HTML avec JavaScript.
- Les événements : `click`, `load`, `animationend`.
- `setTimeout` : exécuter une action après un petit délai.
- Les classes CSS pilotées par JavaScript : `add`, `remove`.
- `innerHTML` et `textContent` : comprendre quand on veut garder du HTML.

Conseil : ne cherche pas à rendre ce script beaucoup plus abstrait maintenant. Il est plus important que tu comprennes bien le parcours des données : clic -> choix de la section -> choix du contenu -> animation -> affichage.